

GOTabPFN: From Feature Ordering to Compact Tokenization for Tabular Foundation Models on High-Dimensional Data

Al Zadid Sultan Bin Habib¹ Md Younus Ahamed² Prashnna Kumar Gyawali³
Gianfranco Doretto⁴ Donald A. Adjeroh⁵

^{1,2,3,5}Lane Department of Computer Science and Electrical Engineering,
West Virginia University, Morgantown, WV 26506, USA

⁴Scientific Computing and Imaging Institute & Department of Biomedical Informatics,
University of Utah, Salt Lake City, UT 84112, USA

Corresponding author: ¹ah00069@mix.wvu.edu

ICML 2026

Why GOTabPFN?

HDLSS tabular data

High-Dimensional, Low-Sample Size means:

$$m \gg n$$

where m is the number of features and n is the number of samples.

TabPFN bottleneck

TabPFN-style models are strong general-purpose predictors, but direct use becomes difficult when m is very large.

$$m \rightarrow \text{too many tokens}$$

Goal: make frozen TabPFN-style prediction practical for high-dimensional tabular data by turning many raw features into fewer structured meta-features.

Core Idea: Order, Compress, Predict

1. Order

Use GO-LR to place related features close together on a structured feature axis.

2. Compress

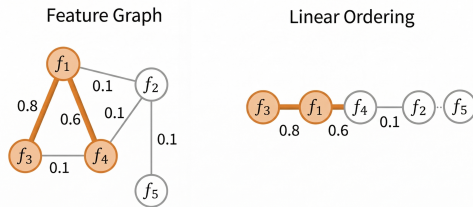
Use NSC to pool local ordered neighborhoods into compact meta-features.

3. Predict

Use the compact representation with a frozen TabPFN-2.5 prediction head.

Pipeline: Raw high-dimensional table $\rightarrow$ GO-LR $\rightarrow$ NSC $\rightarrow$ Frozen TabPFN-2.5

GO-LR: Graph-guided Ordering with Local Refinement



Goal: Place connected nodes on a line such that the most strongly related nodes are close to each other

Input

Build a weighted graph where nodes are features and edges encode feature relationships.

Goal

Place strongly related features close together on a one-dimensional feature axis.

Output: an ordered feature sequence for NSC compression.

GO-LR: Optimization View

Local objective

GO-LR minimizes a MinLA-style dispersion cost:

$$D_G(\pi) = \sum_{(i,j) \in E} w_{ij} |\pi(i) - \pi(j)|$$

Large-weight feature pairs are penalized more when placed far apart.

Practical solver

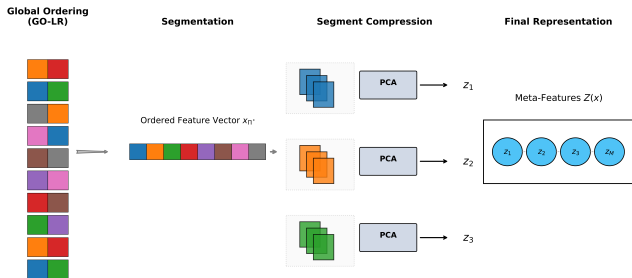
GO-LR uses:

- TSP-path-style initialization
- direction selection
- local adjacent-swap refinement

Result: one global order Π^* .

The ordered axis is passed to NSC, where nearby features are compressed together.

NSC: Neuro-Inspired Subunit Compression



Segmentation

After GO-LR, the ordered feature vector is partitioned into contiguous local segments.

Compression

Each segment is compressed into one meta-feature, producing a smaller structured representation.

Output: a compact meta-feature vector $Z(x) = (z_1, z_2, \dots, z_M)$.

NSC: From Ordered Features to Meta-Features

Ordered input

After GO-LR, features lie on a structured axis:

$$x^\Pi = (x_{\Pi^*(1)}, \dots, x_{\Pi^*(m)})$$

NSC groups nearby ordered features into local segments

$$\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_M.$$

Segment compression

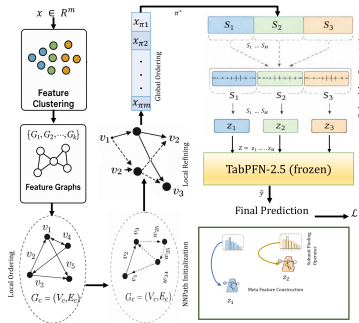
For each segment t , one meta-feature is computed as:

$$z_t(x) = (u_t(x) - \mu_t)^\top v_t$$

where $u_t(x) = x_{\mathcal{S}_t}^\Pi$. This reduces m raw features to M compact meta-features, with $M \ll m$.

These compact tokens are then passed to the frozen TabPFN-2.5 prediction head.

End-to-End GOTabPFN Pipeline



GO-LR

Orders raw features into a structured feature axis.

NSC

Compresses local ordered neighborhoods into meta-features.

TabPFN-2.5

Predicts from compact tokens using a frozen backbone.

Design: adapt the input, not the backbone.

Dataset Overview

Primary Biomedical HDLSS

Dataset	Samples	Features	Classes
Arcene	200	10,000	2
Colon	62	2,000	2
Prostate-GE	102	5,966	2
ALLAML	72	7,129	2
SMK-CAN-187	187	19,993	2
Lung	203	3,312	5
TOX-171	171	5,748	4
GLI-85	85	22,283	2

All datasets satisfy the HDLSS condition:

$$m \gg n$$

They contain only 62–203 samples but up to 22,283 features.

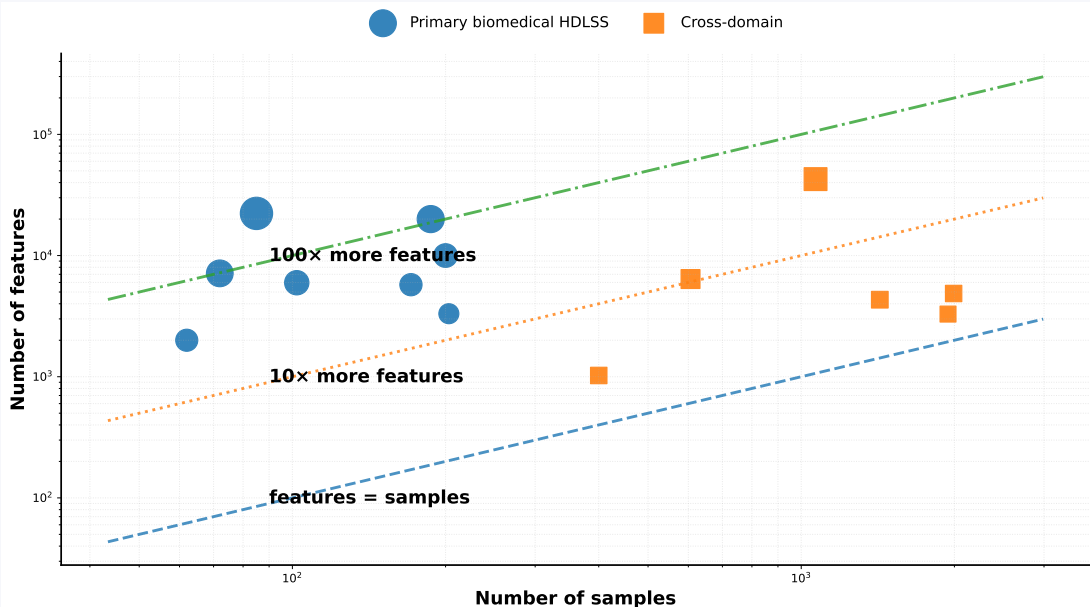
Cross-Domain High-Dimensional Data

Dataset	Samples	Features	Classes
ORL	400	1,024	40
DrivFace	606	6,400	7
PCMAC	1,943	3,289	2
RELATHE	1,427	4,322	2
BASEHOCK	1,993	4,862	2
CIFAR-10	11,000	2,048	10
Cell Cycle	1,067	42,729	3

These datasets test generalization beyond the primary biomedical setting.

They include image-derived, text-like, sensor, and biological features.

Goal: evaluate GOTabPFN on extreme biomedical HDLSS data and diverse high-dimensional domains.



How HDLSS Differs from Normal Tabular Data

Typical Tabular Data

Many samples, fewer features

$$n \gg m$$

HDLSS Data

Few samples, many features

$$m \gg n$$

Notation

m : number of features

n : number of samples

HDLSS differs sharply from widely used tabular benchmarks

Benchmark	HDLSS Focus	Feature Range m	Sample Range n	Min m/n	Max m/n
OpenML-CC18	No	5–3,073	500–96,320	1.56×10^{-4}	0.79
OpenML-CTR23	No	6–117	517–72,000	1.85×10^{-5}	0.11
Grinsztajn et al.	No	4–613	2,554–940,160	1.52×10^{-5}	0.09
TabZilla	No	5–4,297	148–1,025,009	1.07×10^{-5}	0.79
TALENT	No	3–970	395–991,346	1.63×10^{-5}	0.28
TabArena	No	5–1,777	748–150,000	7.33×10^{-5}	0.47

Key difference: conventional benchmarks generally satisfy $m/n < 1$, while HDLSS datasets can contain more than 262 features per sample.

Experimental Protocol

Repeated Cross-Validation

5-fold CV $\times$ 5 repetitions

25 evaluation splits per
dataset

Why Repeated CV Matters in HDLSS

When $n \ll m$, results can be highly sensitive to the training partition.

Repeated evaluation provides a more reliable estimate of:

- predictive performance
- split-to-split variability
- robustness under limited samples

Reported result = Mean $\pm$ Standard Deviation

Hyperparameter Tuning Protocol

Frozen Foundation Models

TabDPT and TabICL are used directly.

They do not require dataset-specific hyperparameter tuning in our evaluation.

Classical / Neural Baselines

Models such as XGBoost and MLP are sensitive to design choices.

Therefore, they require hyperparameter search.

GOTabPFN

Only the input-adaptation modules are tuned:

GO-LR ordering and NSC compression.

The TabPFN-2.5 predictor remains frozen.

Fairness rule: all tunable models use the same tuning budget, while frozen foundation predictors are evaluated without additional dataset-specific tuning.

Comparison Against 55 Baselines

Classical Machine Learning

- Lasso
- SVM
- XGBoost

Linear, kernel-based, and tree-ensemble baselines.

Deep Tabular Learning

- MLP
- TabM
- RealMLP

Neural baselines for learning nonlinear tabular representations.

HDLSS-Specific Methods

- ProtoGate
- LLSPIN
- STG

Methods designed for feature selection and low-sample, high-dimensional learning.

Tabular Foundation Models

- TabPFN-Wide
- TabDPT
- TabICL

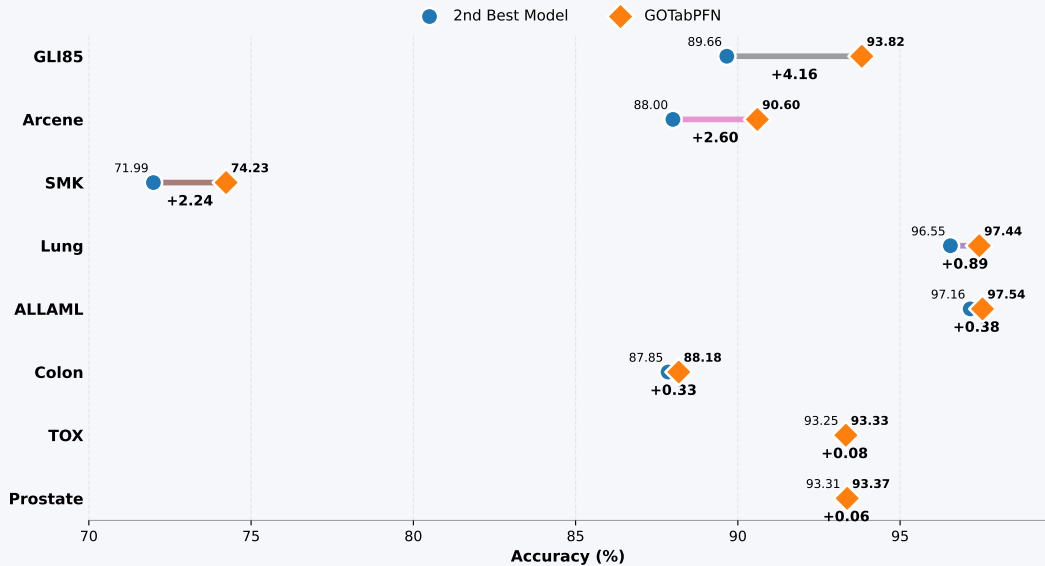
Recent foundation models for in-context tabular prediction.

55 baselines in total across four model families; the main table highlights the strongest competitors.

HDLSS Classification Results

Model	COL	LNG	GLI	SMK	AML	PRS	ARC	TOX	Rank
GOTabPFN	88.18±10.05	97.44±2.32	93.82±5.81	74.23±5.17	97.54±3.86	93.37±4.48	90.60±3.97	93.33±4.74	1.00±0.00
TANDEM	86.15±7.75	96.46±2.88	91.53±6.02	72.72±5.69	95.81±5.53	91.55±4.32	86.90±6.34	93.08±2.61	3.63±1.32
TabPFN-W	87.85±7.28	96.55±2.15	88.47±5.75	68.78±8.60	97.16±4.10	93.10±5.92	88.00±5.20	89.35±4.95	3.75±2.38
TabDPT	86.26±7.27	96.05±2.57	87.76±5.86	71.99±7.32	96.32±4.15	90.94±5.72	82.10±6.48	93.25±3.44	4.88±1.69
TabICL	84.62±10.52	96.36±2.61	87.06±6.23	68.73±7.28	95.52±5.59	90.17±5.93	82.60±6.14	88.78±5.92	7.63±2.29
BETA	84.73±9.36	94.38±3.34	86.21±8.91	70.21±5.61	95.67±7.54	87.53±4.67	86.45±5.92	90.38±6.42	8.13±4.31
TuneTables	86.80±2.14	94.37±2.35	89.66±3.12	70.28±6.46	95.80±2.14	93.31±2.83	81.40±3.66	77.96±2.55	8.38±7.70
Lasso	79.40±10.18	94.47±4.39	85.88±4.71	61.19±13.72	87.24±3.39	91.18±6.39	81.00±3.39	91.86±6.03	11.13±5.06
MLP	83.95±9.80	96.47±2.69	85.41±8.00	59.05±7.44	89.98±9.17	89.20±6.07	78.40±4.05	92.48±4.28	11.63±5.45
ProtoGate	83.95±9.82	93.44±6.37	82.48±5.68	60.16±5.10	86.12±3.34	90.58±5.72	81.50±5.10	92.34±5.67	12.06±4.77

GOTabPFN ranks first on all eight HDLSS datasets, achieving an average rank of **1.00 ± 0.00** under 5×5 repeated cross-validation.



Results from Cross-Domain Data

Model	ORL	BAS	REL	PCM	CCY	CIF	DF-R	DF-C	Rank
GOTabPFN	100.00±0.00	97.11±1.00	88.87±1.32	89.51±2.24	79.94±2.53	88.45±0.89	0.6548±0.0992	86.70±2.48	1.25±0.66
TabDPT	97.32±0.68	97.00±0.68	88.26±1.49	87.58±0.96	77.52±2.44	88.00±0.40	0.6505±0.0820	83.57±3.04	4.44±1.49
TabPFN-W	96.10±6.52	97.04±0.72	87.84±3.90	88.56±0.71	78.67±2.42	88.12±0.60	0.6430±0.0772	85.42±2.66	3.88±1.62
TuneTables	93.14±1.02	93.14±1.02	88.26±2.31	84.27±3.06	78.30±1.13	78.05±0.15	0.6332±0.0675	84.62±2.43	5.94±1.91
TabICL	99.20±1.87	96.84±0.75	88.15±1.68	88.68±1.94	–	87.61±1.00	–	85.55±2.45	5.31±2.46
TANDEM	99.00±2.45	96.72±0.86	89.42±1.42	88.99±1.33	77.31±1.97	87.80±0.36	0.6488±0.0770	84.42±2.55	4.00±1.94
Lasso	96.00±3.82	96.74±0.88	86.87±1.46	88.85±1.80	77.86±2.42	86.42±1.10	0.3194±0.0668	77.95±3.03	6.13±1.69
MLP	91.00±8.37	96.91±1.15	89.12±1.85	87.71±1.95	76.38±1.42	88.15±0.54	0.5682±0.1258	80.59±3.72	5.25±2.17
ProtoGate	66.40±11.32	–	58.40±3.15	–	–	–	−0.1636 ± 0.0738	66.70±10.20	8.81±0.35

GOTabPFN achieves the best average rank and leads on 7 of the 8 cross-domain tasks.

Takeaway, Resources, and Acknowledgment

GOTabPFN makes frozen TabPFN-style prediction practical for high-dimensional tabular data by combining **feature ordering** with **compact tokenization**.

Project links

ICML: icml.cc/virtual/2026/poster/62523

Project: zadidhabib.com/gotabpfn.html

GitHub: github.com/zadid6pretam/GOTabPFN

Try or ask

Demo: ZeroGPU [zadid6pretam/GOTabPFN](https://zadid6pretam.com/GOTabPFN/); CPU
Backup [zadid6pretam/GOTabPFN_CPU](https://zadid6pretam.com/GOTabPFN_CPU/)

Install: `pip install gotabpfn`

Contact: ah00069@mix.wvu.edu

Acknowledgment: This work was supported in part by the U.S. National Science Foundation (NSF) Awards #1920920, #2125872, and #2223793. We thank the anonymous ICML reviewers for their constructive feedback.

Thank You!

Questions?

`github.com/zadid6pretam/GOTabPFN`

`ah00069@mix.wvu.edu`

`zadidhabib.com/gotabpfn.html`